AI6101 Reinforcement Learning Assignment

Tabular Q-learning for the CliffBoxPushing Grid World

Name: HUANGTING Matriculation number: G2605438E

1 Problem and overview

The agent must push a box from $(1, 1)$ to the goal $(4, 13)$ on the fixed 6×14 map without the agent or the box entering a cliff cell. The state is $s = (a_r, a_c, b_r, b_c)$ (agent and box positions), the actions are up/down/left/right, and the episode ends at the cliff, at the goal, or after 100 steps. The reward of each step is

$$r_t = -1 - \|a_t - b_t\|_1 - \|b_t - g\|_1 \quad (+1000 \text{ if the box reaches the goal}, -1000 \text{ if the agent or box enters a cliff}).$$

Two features of `step()` make the task hard: the box can never be on row 0, row 5 or column 0, and the only feasible corridor passes right next to several cliffs. The box has to be dropped through the single gap in column 3, lifted over the column-6 cliff, slid along row 1 and pushed down column 13.

I implemented **tabular Q-learning** (`RLAgent`), the V-table/policy visualisation, and the action-noise robustness study, plus both bonus tasks (exploration strategies; sparse reward). The game itself (map, reward, `step()`) is never modified. Section 7 lists every change made to the agent and the training/evaluation loops. All numbers below come from the submitted notebook.

2 Method: Q-learning

Q-learning [1] is off-policy TD control. After each transition (s, a, r, s'),

$$Q(s,a) \leftarrow Q(s,a) + \alpha[y - Q(s,a)], \quad y = \begin{cases} r & \text{if } s' \text{ is a cliff or goal state} \\ r + \gamma \max_{a'} Q(s', a') & \text{otherwise.} \end{cases}$$

The behaviour policy is ε-greedy with random tie-breaking. Hyper-parameters: $\alpha = 0.1, \gamma = 0.99, Q_0 = 0$, 15,000 episodes, and ε decaying linearly from 1.0 to 0.01 over the first half of training. Three implementation details matter:

- **Time-outs are truncations, not terminal states.** Time is not part of the state, so treating the 100-step cut-off as terminal would wrongly teach the agent that some ordinary states have zero future value. The notebook therefore bootstraps at a time-out and stops bootstrapping only at the cliff or the goal (partial-episode bootstrapping [2]).
- **$Q_0 = 0$ is optimistic.** Every non-terminal reward is negative, so untried actions look better than tried ones. This drives systematic exploration on top of ε-greedy (Section 5 isolates this effect).
- **Action noise is part of the environment.** For the robustness study, the agent picks a, the loop may replace it with another action, and the update is applied to the **chosen** a. $Q(s, a)$ therefore estimates the value of choosing a under noise, which is what a noise-aware policy needs.

Sanity check. The reachable state space is small (2,739 non-terminal states), so the notebook also solves the 100-step problem exactly by finite-horizon dynamic programming. This is used **only as a reference**, never for training. The optimal return is **577** (39 steps). With action noise p, the same DP gives the best achievable expected return and the success probability of that return-maximising policy (Table 1). The expected return is an upper bound for every policy. The success probability is only a reference value: the DP maximises return, not success, so another policy could in principle succeed slightly more often.

Table 1: Exact optimum of the 100-step problem (finite-horizon DP over the 2,739 reachable states; reference only).

Action noise p	0	0.1	0.2
Optimal expected return	577.0	−278.6	−942.9
Success probability	1.000	0.587	0.252

3 Results on the original task

3.1 Learning progress

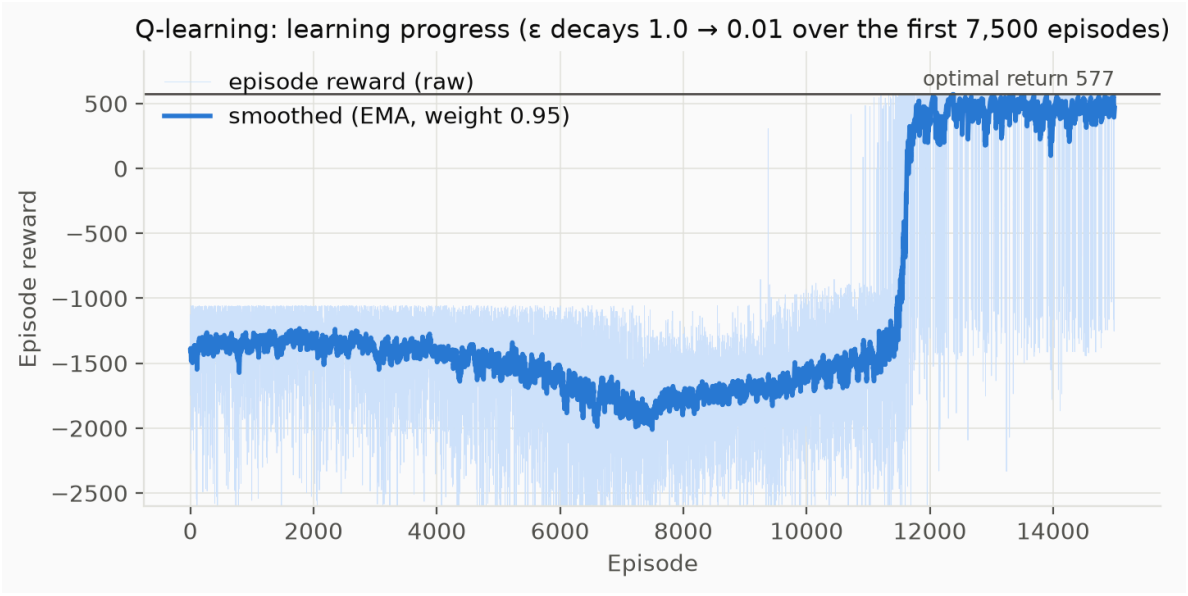


Figure 1: Episode reward vs. episode for the Q-learning agent (light: raw, dark: EMA-smoothed with weight 0.95).

Figure 1 has three phases. **Random exploration (episodes 0–≈5k).** ε is still large, so most episodes end at a cliff after a few steps (≈ −1,000 plus the distance penalties) or run into the time-out (≈ −1,500 to −2,500). The smoothed reward stays near −1,400. **Cliff avoidance without the goal (≈5k–11.5k).** As ε falls to its floor (reached at episode 7,500), the agent stops falling and survives the full 100 steps. Without the goal, this costs **more** reward than dying early, so the curve dips to about −1,900. It then climbs slowly as the optimistic Q-table pulls the box further along the corridor (smaller box-to-goal penalty). The goal is reached for the first time in episode 9,376. **Convergence (≈11.5k–15k).** Once the +1000 has propagated back through the bootstrapped targets, the reward jumps to the optimum of 577 within a few hundred episodes. Over the last 2,000 training episodes ($\varepsilon = 0.01$), 93.9% of the episodes reach the goal. The remaining dips are caused by the occasional random action.

3.2 Learned policy and V-table

The template's `visualize()` was completed with $V(s) = \max_a Q(s,a)$ and $\pi(s) = \arg\max_a Q(s,a)$. It prints the 20 largest values, which all belong to states with the box one or two pushes from the goal. Because the state is 4-D, I show the policy in three complementary ways.

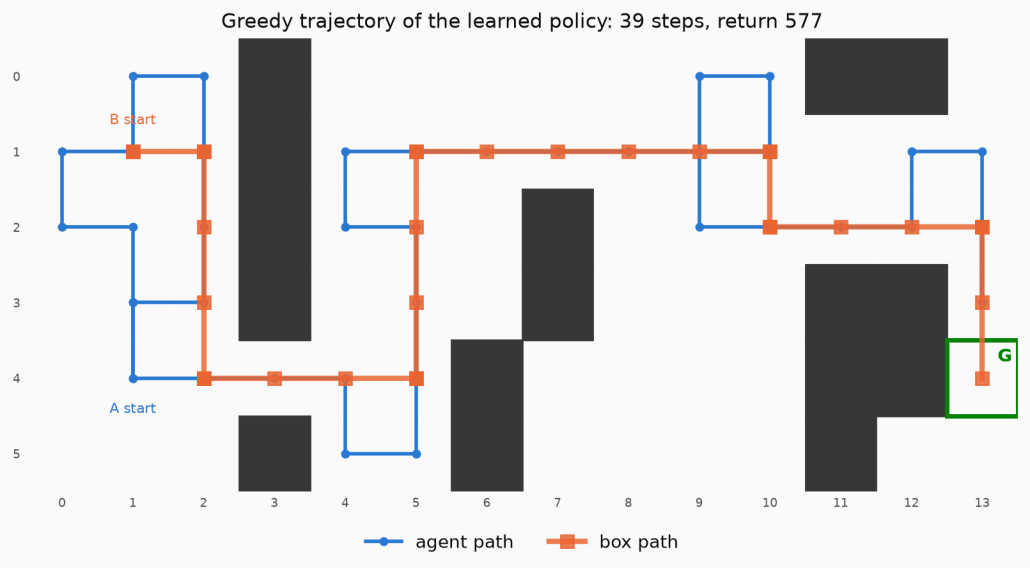


Figure 2: Greedy trajectory of the learned policy (no exploration, no noise): 39 steps, return 577, equal to the DP optimum. The box is pushed right, down through the gap at $(4, 3)$, right along row 4, up column 5, right along row 1, down to row 2 at column 10, right to column 13 and finally down into the goal.

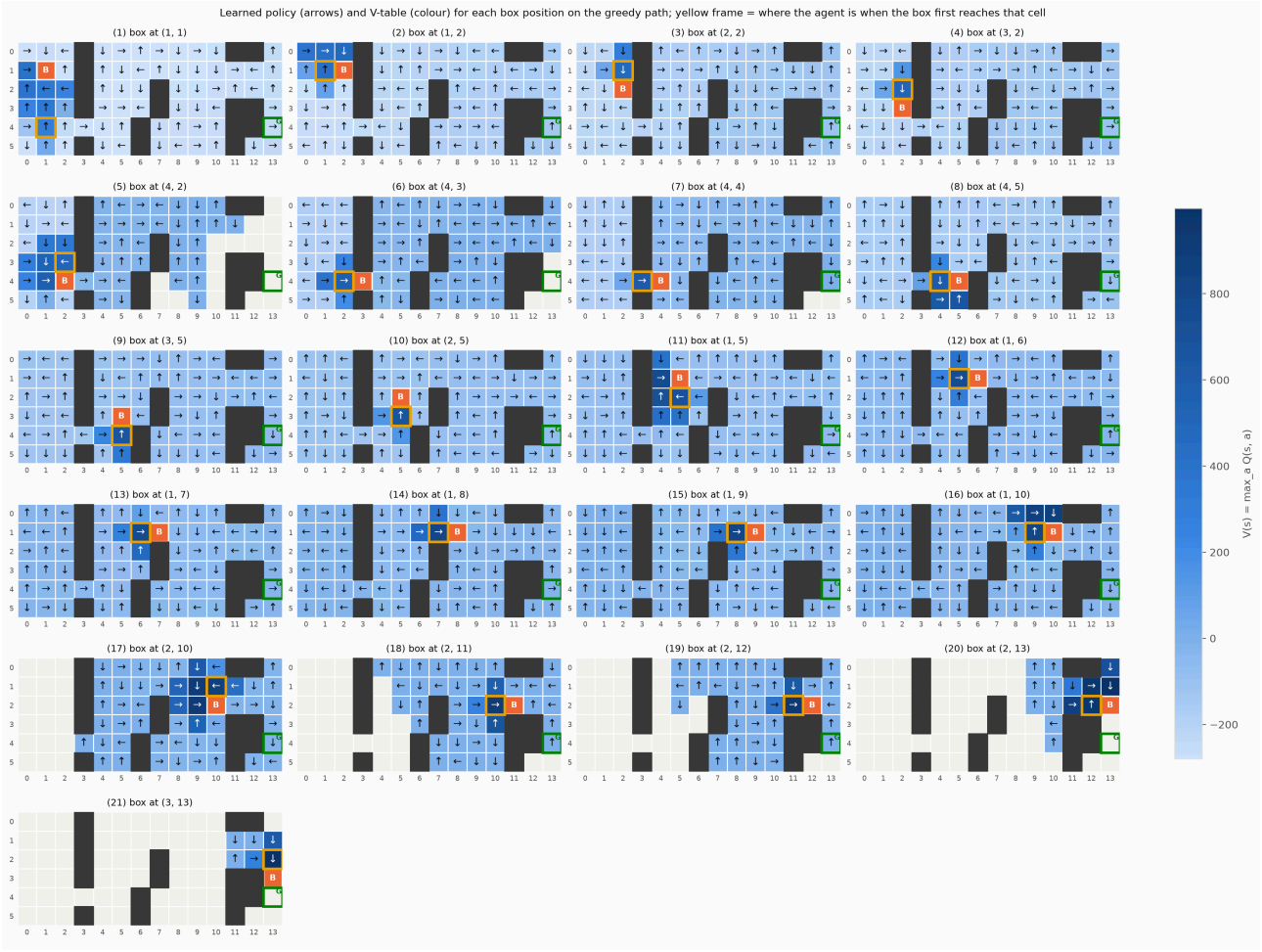


Figure 3: Policy maps. One panel per box position visited by the greedy trajectory. Each agent cell is coloured by $V(s)$, the arrow is the greedy action $\pi(s)$, grey cells were never visited, and the yellow frame is where the agent stands when the box first enters that cell.

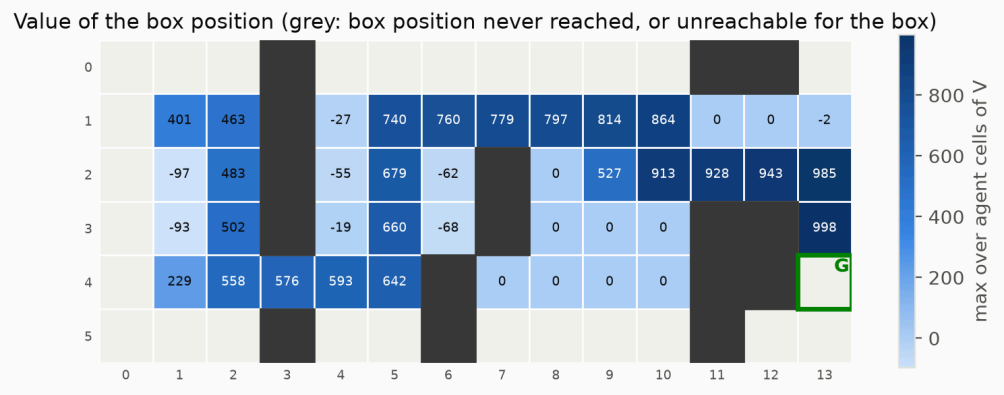


Figure 4: Value of each box position: $\max_{\text{agent cell}} V(\text{agent}, \text{box})$. Grey: never reached.

The policy maps show the structure of the solution. The yellow frame is the cell from which the agent has just pushed the box. From there, the arrows lead the agent around the box to the side needed for the next push without entering a cliff. For example, in panel 2 the agent goes up, right, and then pushes the box down. Arrows far from the path belong to rarely visited states and matter less. Along the path, the box value in Figure 4 rises monotonically from 401 at the start to 998 one push before the goal. Box positions that lead into dead ends (e.g. row 3–4 in columns 7–10, from where the box cannot be brought back to row 1–2) keep a value close to 0. Greedy evaluation with the template's evaluation cell (1,000 episodes, $\varepsilon = 0$) gives a success rate of **100%**. The environment and the policy are deterministic, so every episode repeats the optimal 39-step trajectory.

4 Action-noise robustness study

Protocol. With probability p, the selected action is replaced by one of the other three actions uniformly at random. This happens in the training/evaluation loop; `step()` is unchanged. For each $p_{\text{train}} \in \{0, 0.1, 0.2\}$ I trained **five seeds** with exactly the main agent's configuration, under two budgets: the template's 15,000 episodes and a long budget of 300,000 episodes. Every policy was evaluated greedily ($\varepsilon = 0$, only the specified noise) for **1,000 episodes** under each $p_{\text{test}} \in \{0, 0.1, 0.2\}$.



Figure 5: Training curves under action noise (mean over 5 seeds; band: min–max over seeds; moving average over 2% of the budget). Top: episode reward. Bottom: training success rate.

How action noise affects learning. With the template's 15,000 episodes, all five noise-free agents converge (first success after 10.7k–11.1k episodes), but **no noisy agent reaches the goal even once during training**. Their greedy policies therefore have a success rate of 0 under every p_{test} (Figure 6, top). The reason is that noise breaks the optimism-driven exploration that solves the noise-free task. Reaching the goal for the first time requires pushing the box ≈15 cells along a cliff-lined corridor. With $p = 0.1$, each step has a 10% chance of a random slip, and near the corridor many slips send the agent or the box into a cliff. The −1000 is credited to the action that was **chosen**, so corridor states quickly look much worse than the "safe" start region. The agent learns to avoid the corridor before it ever finds the +1000, and settles on surviving until the time-out. With 300,000 episodes, all three settings eventually learn (Figure 5, right), but noise delays the first success from ≈112k episodes to ≈141k–161k ($p = 0.1$) and ≈160k–219k ($p = 0.2$). In the 300k run, ε reaches its floor at episode 150k. The final training success rates (≈0.96, ≈0.50, ≈0.18) stay below the DP values in Table 1, partly because training still explores with $\varepsilon = 0.01$ and partly because of the policy instability discussed below. Under noise, even the optimal policy fails 41% ($p = 0.1$) or 75% ($p = 0.2$) of the time, because 39 consecutive moves next to cliffs have to survive the random slips. Low success rates under noise are therefore mostly a property of the task, not a learning failure.

Table 2: Long budget (300,000 episodes). The median seed is reported because one seed of each noisy setting fails (see text).

p_{train}	First training success (episode, 5 seeds)	Success rate, mean of 5 seeds p_{test} = 0 / 0.1 / 0.2	Success rate, median seed p_{test} = 0 / 0.1 / 0.2	Return at $p_{\text{test}} = 0.1$ (median seed)
0	110.8k – 113.5k	1.00 / 0.49 / 0.17	1.00 / 0.50 / 0.17	−408
0.1	140.9k – 161.0k	0.80 / 0.54 / 0.23	1.00 / 0.58 / 0.25	−305
0.2	160.0k – 219.1k	0.80 / 0.45 / 0.18	1.00 / 0.55 / 0.23	−345
DP policy	—	1.00 / 0.59 / 0.25	1.00 / 0.59 / 0.25	−279

Cross-noise generalisation. The noise-free policy is perfect at $p_{\text{test}} = 0$ but drops to 0.49 and 0.17 success at $p_{\text{test}} = 0.1$ and 0.2, which is 84% and 67% of the DP policy's success probability. It only knows the shortest route, which hugs the cliffs, and its values for off-path states (where a slip lands it) were learned only through rare ε-

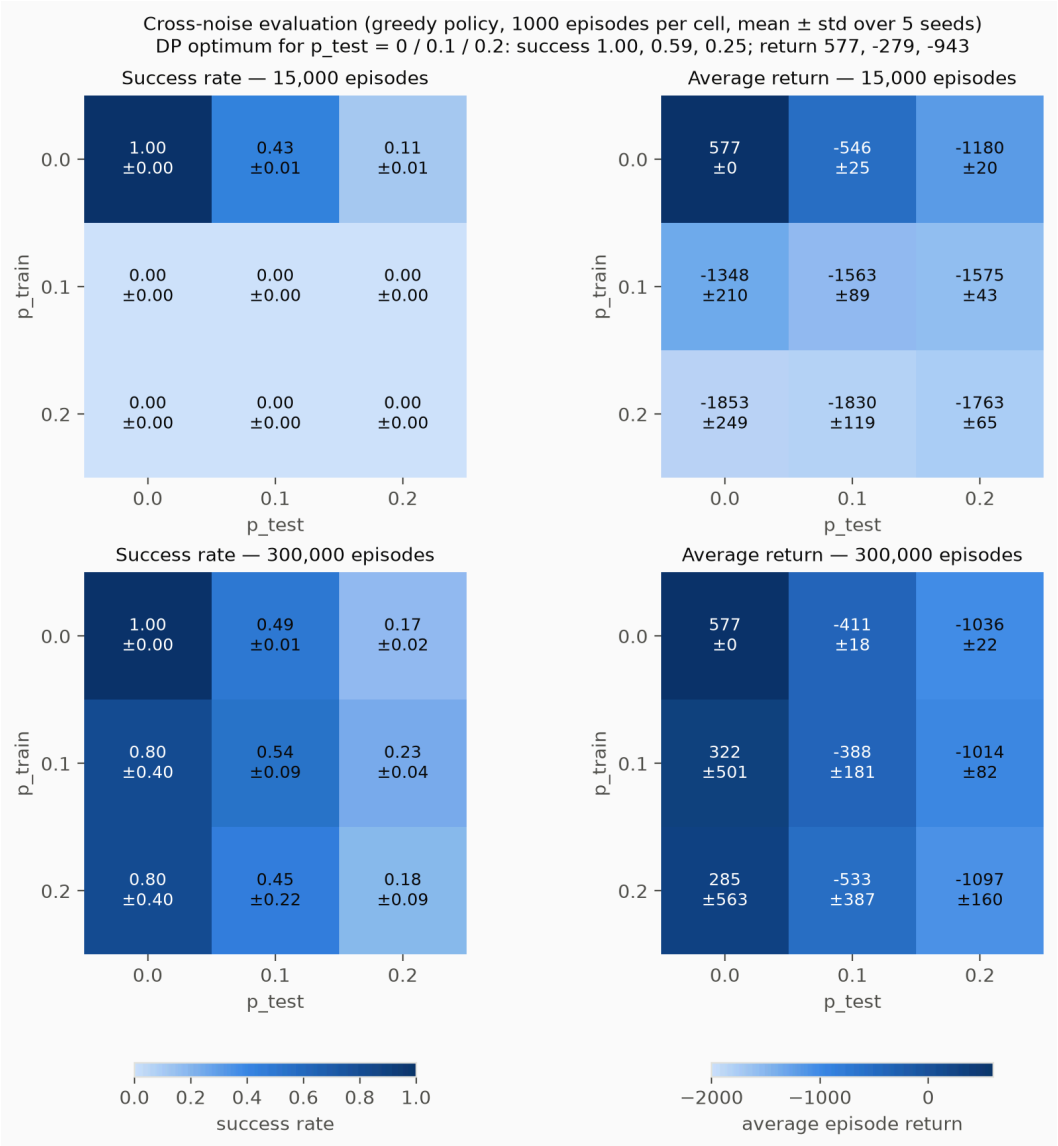


Figure 6: Cross-noise evaluation matrices (rows p_{train}, columns p_{test}; mean ± std over 5 seeds, 1,000 episodes per cell). Top: 15,000 training episodes. Bottom: 300,000.

exploration. Training with moderate noise ($p_{\text{train}} = 0.1$) gives the most robust policies. Four of the five seeds reach 0.575–0.586 success at $p_{\text{test}} = 0.1$ and 0.246–0.257 at $p_{\text{test}} = 0.2$, i.e. 98–102% of the DP policy's success probability (values above 100% are within the ±0.016 standard error of a 1,000-episode estimate). Their return at $p_{\text{test}} = 0.1$ (−283 to −307) is also close to the optimum of −279. They still solve the noise-free task, with a return of 571–573 instead of 577: the same 39-step length, but the agent keeps a slightly different position relative to the box. Training with $p = 0.1$ helps in two ways. The learning target includes the consequences of slips, so actions whose slips lead into a cliff are valued lower. The slips also make the agent visit off-path states much more often, so it learns how to recover from them. Even averaged over all five seeds (including the failed one), $p_{\text{train}} = 0.1$ beats $p_{\text{train}} = 0$ at both noisy test levels, in success rate (0.54 vs 0.49, 0.23 vs 0.17) and in return (−388 vs −411, −1014 vs −1036). Heavier noise does not help further. $p_{\text{train}} = 0.2$ is worse than $p_{\text{train}} = 0.1$ even when **tested** at $p = 0.2$ (median 0.23 vs 0.25). It finds the goal later (on average at episode 184k vs 151k), so its values are less converged at the end of the budget, and the noisier targets make the constant-step-size estimates more variable. One $p_{\text{train}} = 0.2$ seed also takes a 41-step detour (return 550) on the noise-free task, a visibly more cautious route.

Why one seed per noisy setting fails. Seed 1 fails the noise-free task at the end of training for both $p_{\text{train}} = 0.1$ and 0.2 (100 steps, return −679 and −840), which lowers the 5-seed means in Table 2. To find out whether these runs failed to learn, the notebook stores a Q-table snapshot every 10,000 episodes over the last third of the long runs and evaluates each one greedily (Figure 7). The failure turns out to be an unlucky final snapshot, not a failure to learn. The training success of seed 1 over the last third (0.496 at $p = 0.1$, 0.162 at $p = 0.2$) is the same as that of the other seeds (0.485–0.503 and 0.091–0.156), and 8 of its 11 snapshots at $p_{\text{train}} = 0.1$ reach ≈0.50–0.62 success at

$p_{\text{test}} = 0.1$. In turn, the seeds that end well have bad snapshots. With $p_{\text{train}} = 0.1$, 18% of all 55 snapshots fail the noise-free task, and every seed has at least one snapshot with ≤ 0.40 success at $p_{\text{test}} = 0.1$ (seed 0 drops to 0.00 at 280k episodes). With $p_{\text{train}} = 0.2$, only 56% of the snapshots solve the noise-free task, and without noise 100% do. The cause is the constant step size $\alpha = 0.1$. Under noise, the TD target of the same (s, a) is random. A slip into a cliff gives a target near −1000, far below the target after a normal step, so a single update moves $Q(s, a)$ by $\alpha \cdot 1000 \approx 100$ or more. This can exceed the gap between the two best actions at a critical state. The greedy action there keeps flipping, and when it flips to a bad action, the deterministic noise-free rollout gets stuck until the time-out. Noise-free targets are deterministic, so those values converge. Standard remedies, not applied here so that every p_{train} uses the main agent's exact configuration, are a decaying step size ($\alpha \propto 1/N(s, a)$, which satisfies the Robbins–Monro conditions [3]) or selecting the final policy by evaluating checkpoints on separate validation episodes. The conclusion does not depend on picking good snapshots. Averaged over **all** snapshots, including the bad ones, $p_{\text{train}} = 0.1$ still beats $p_{\text{train}} = 0$ at $p_{\text{test}} = 0.1$ (0.512 vs 0.495) and at $p_{\text{test}} = 0.2$ (0.209 vs 0.173). $p_{\text{train}} = 0.2$ is worse at every test level (0.564 / 0.322 / 0.124).

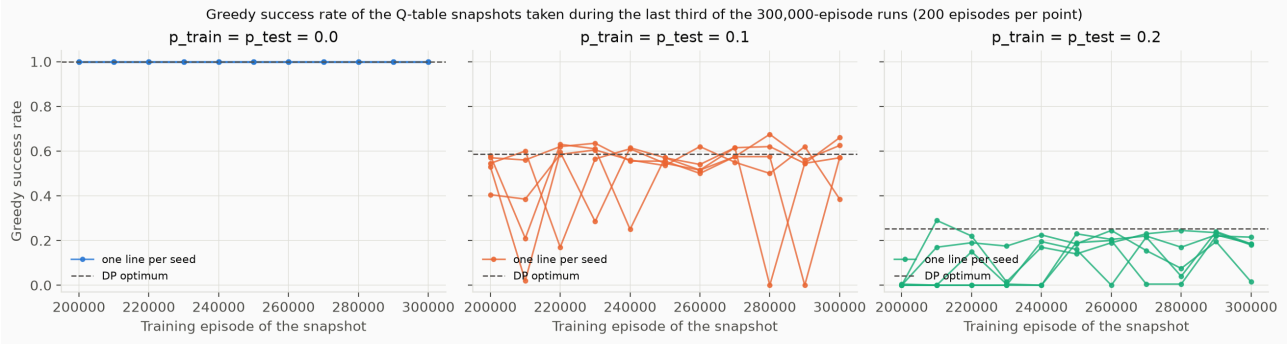


Figure 7: Greedy success rate ($p_{\text{test}} = p_{\text{train}}$, 200 episodes per point) of the Q-table snapshots taken every 10,000 episodes during the last third of the 300,000-episode runs. One line per seed; dashed: success probability of the DP policy.

Answer. Action noise makes learning much harder: it needs about 20× the template's budget, it finds the goal later, and with a constant step size its greedy policy is unstable from one checkpoint to the next. Training with moderate noise ($p_{\text{train}} = 0.1$) does improve robustness. A converged policy closes nearly the whole gap to the optimal noisy policy while losing almost nothing on the noise-free task, and even averaged over all checkpoints it beats noise-free training under noise. Training with too much noise ($p_{\text{train}} = 0.2$) brings no additional benefit within the budget.

5 Bonus A: exploration strategies

All agents share the Q-learning update, α, γ, the 15,000-episode budget and three seeds (no action noise). Only the behaviour policy changes: decaying ε-greedy (main agent), constant $\varepsilon = 0.1$ (the template default), UCB1 [4] ($\arg\max_a Q(s, a) + c\sqrt{\ln N(s)/N(s, a)}$, $c = 10$, untried actions first), and **no exploration** (always greedy) with the default $Q_0 = 0$ or with a pessimistic $Q_0 = -3000$.

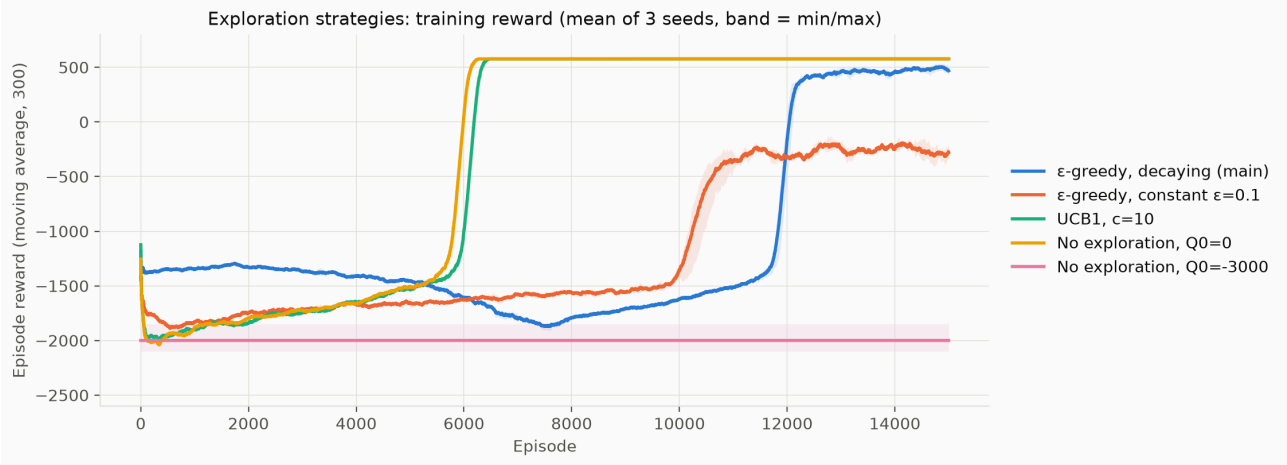


Figure 8: Training reward of the exploration strategies (moving average over 300 episodes, mean of 3 seeds, band: min–max).

Table 3: Exploration strategies (mean over 3 seeds). SR: success rate. Greedy return: one $\varepsilon = 0$ episode of the final policy.

Strategy	Goal found	First success (ep.)	Mean training reward	Training SR (all)	Training SR (last 1k)	Greedy return
ε-greedy, decaying (main)	3/3	10,921	−1,106	0.203	0.949	577
ε-greedy, constant 0.1	3/3	7,924	−1,213	0.185	0.571	577
UCB1, $c = 10$	3/3	4,526	−330	0.605	1.000	577
No exploration, $Q_0 = 0$	3/3	4,719	−301	0.618	1.000	577
No exploration, $Q_0 = -3000$	0/3	—	−2,000	0.000	0.000	−2,000

Optimism-driven strategies win clearly. UCB1 and the greedy agent with $Q_0 = 0$ find the goal after ≈4.5k–4.7k episodes, more than twice as fast as ε-greedy. They reach the optimal return by episode ≈6.3k and stay there exactly, with roughly 3.5× less training cost (mean reward ≈ −300 vs ≈ −1,100 to −1,200). **"No exploration" with $Q_0 = 0$ is not exploration-free.** Every real reward is negative, so 0 is an optimistic estimate (optimism in the face of uncertainty [3]). Each time a state-action is tried, its value drops below the untried ones, so the greedy agent sweeps the state space systematically. The pessimistic control proves this: with $Q_0 = -3000$, the first update **raises** the tried action above −3000 ($-3000 + \alpha(r + 30) > -3000$ for $r > -30$). The agent then repeats that action for 100 steps in every episode (return −2,000 each time) and never finds the goal. **UCB1** turns uncertainty into an explicit, shrinking bonus, so its exploration is directed at rarely tried actions. With $c = 10$, the bonus is small next to the existing $Q_0 = 0$ optimism, and UCB1 behaves almost like the optimistic greedy agent. In preliminary runs, a bonus on the reward scale ($c = 100$–500) over-explored and converged later or not at all. **ε-greedy explores by undirected dithering, which suits this task poorly.** A success needs ≈40 consecutive correct actions next to cliffs, and one random action can push the box over the edge. With decaying ε, the goal is found only after ε has reached its floor (10.9k vs 7.5k episodes), so the high-ε phase mostly produces cliff deaths. Constant $\varepsilon = 0.1$ finds the goal earlier (7.9k), but keeps paying for its random actions: only 57% of its last 1,000 training episodes succeed (95% with decaying ε). All strategies except the pessimistic one end with the optimal greedy policy.

6 Bonus B: sparse reward

Here the learning signal is **only** +1000 when the box reaches the goal: no step cost, no distance shaping and no cliff penalty, although falling still ends the episode. Until the first success every update is $0 \to 0$, so an ε-greedy agent (or a greedy agent with $Q_0 = 0$) performs a random walk, and a random walk essentially never moves the box 15+ cells through the cliff corridor.

Idea: deep exploration through optimistic initial values. Initialise $Q_0 > 0$ and act greedily. With zero rewards, the update $Q(s,a) \leftarrow (1-\alpha)Q(s,a) + \alpha\gamma \max_{a'} Q(s',a')$ wears down the value of state-actions that have been tried, while untried ones keep their optimistic value. Through the max in the bootstrapped target, this optimism propagates **backwards**, so the agent deliberately travels multi-step routes to the frontier of unexplored states instead of dithering locally, in the spirit of R-max [5]. Falling off the cliff ends the episode and forfeits all future optimism, so cliff avoidance is learned without any penalty. Once the goal is found, its real +1000 takes over.

Table 4: Sparse-reward results (mean over 3 seeds, 15,000 episodes). Returns are measured with the original reward so that they are comparable with Section 3.

Strategy (sparse reward)	Goal found	First success (ep.)	Training SR (last 1k)	Greedy SR	Greedy return (original reward)
ε-greedy (decaying), $Q_0 = 0$	0/3	—	0.000	0/3	−1,596
Greedy, $Q_0 = 0$	0/3	—	0.000	0/3	−1,377
Greedy, optimistic $Q_0 = 100$	3/3	9,873	1.000	3/3	572
Greedy, optimistic $Q_0 = 1000$	3/3	9,890	0.037	0/3	−1,430

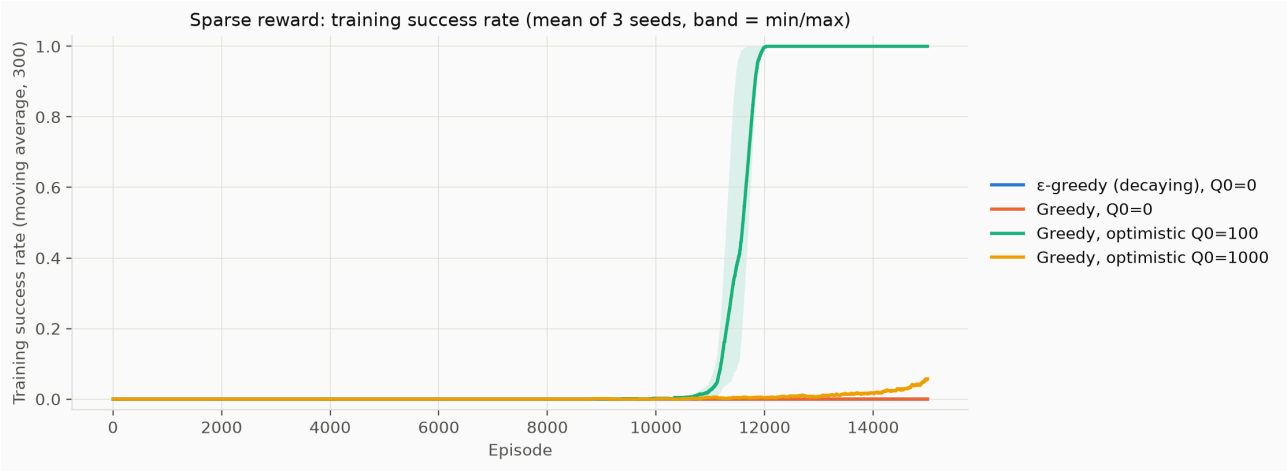


Figure 9: Sparse reward: training success rate (moving average over 300 episodes, mean of 3 seeds, band: min–max).

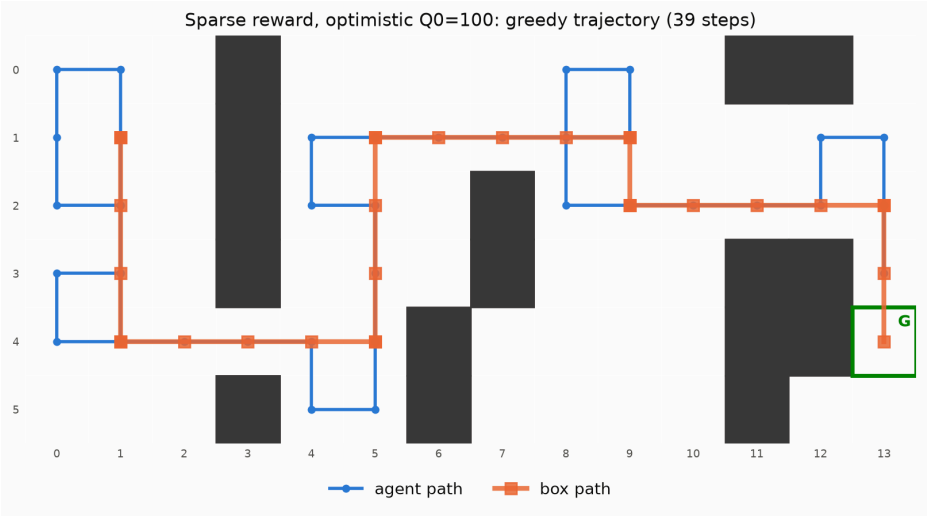


Figure 10: Greedy trajectory of a sparse-reward agent ($Q_0 = 100$, seed 0): 39 steps, original-reward return 569.

The two baselines never see a single success in 15,000 episodes, so their Q-tables stay at zero (0/3 seeds). **Optimistic $Q_0 = 100$ with greedy action selection solves the sparse task for every seed.** The goal is first reached after ≈9.9k episodes. By ≈12k episodes all training episodes succeed, and the greedy policy pushes the box to the goal in 39 steps, the same minimum as the dense-reward optimum (Figure 10). No step cost was needed for this: the discount $\gamma = 0.99$ alone favours the shortest route. Measured with the original reward, the return is 569–573 (mean 572; optimum 577), because the sparse agent does not care how far it stands from the box. **Too much optimism prevents exploitation.** $Q_0 = 1000$ also finds the goal after ≈9.9k episodes but does not converge within 15k episodes (greedy success 0/3). The discounted value of the real solution from the start is only about $1000\gamma^{38} \approx 680$, so untried state-actions anywhere in the state space still look better than the known route, and the agent keeps exploring until that optimism has worn off. The optimistic value must be high enough to beat dead ends (worth 0) but below the value of a real solution.

7 Summary of modifications

None of the following changes the game (map, rewards, `step()`). Their effect is quantified in the sections cited.

Change	Reason / evidence
Linearly decaying ε (1.0 ⟶ 0.01) instead of the template's constant 0.1	Constant $\varepsilon = 0.1$ keeps losing episodes to random cliff steps until the end of training: 57% vs 95% success over the last 1,000 training episodes (Section 5).
Bootstrapping at time-outs	Time is not in the state, so the time limit is a truncation, not a terminal state [2]. A 3-seed ablation in the notebook (Section 5 of the notebook) shows **no measurable effect** on this task (first success 8.9k–11.1k vs 10.8k–11.1k episodes, ≈95% late training success and optimal greedy return in both cases). It is kept because it is the correct target, not for performance.

Action noise applied after the choice; update on the chosen action	Learns the value of choosing a in the noisy MDP, so noise-aware policies are possible (Section 4).
Long budget (300k episodes) for noisy training	With 15k episodes the noisy agents never find the goal (Section 4).
`sparse_reward` flag and `q_init` parameter	Needed for the Bonus B ablation (Section 6).
NumPy compatibility line `np.chararray = np.char.chararray`	NumPy ≥ 2.5 removed the alias used by the provided renderer. The environment code itself is untouched.

8 Conclusion

Tabular Q-learning with a decaying ε-greedy policy solves CliffBoxPushing within the template's 15,000 episodes. The learned greedy policy reaches the goal in 39 steps with return 577, which finite-horizon DP confirms to be optimal. The V-table and the per-box-position policy maps show a consistent solution: the value rises monotonically along the box's route, and in every phase the agent walks around the box to the correct pushing side. Action noise makes the problem much harder. Within 15,000 episodes no noisy agent ever reaches the goal, and with 300,000 episodes learning is slower, and with a constant step size the greedy policy fluctuates between checkpoints. A converged policy trained with moderate noise ($p_{\text{train}} = 0.1$) nevertheless reaches the DP policy's success rate under noise while staying near-optimal without noise, so moderate training noise does improve robustness. Heavier training noise ($p = 0.2$) brings no further benefit. The bonus studies show that what makes exploration work here is **optimism**, not randomness. Optimistic initial values or UCB find the goal more than twice as fast as ε-greedy. With a sparse reward, greedy action selection with a moderately optimistic $Q_0 = 100$ solves the task for every seed, where ε-greedy never succeeds once.

Bibliography

[1] C. J. C. H. Watkins and P. Dayan, "Q-learning," *Machine Learning*, vol. 8, no. 3, pp. 279–292, 1992.

[2] F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev, "Time Limits in Reinforcement Learning," in *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.

[3] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[4] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time Analysis of the Multiarmed Bandit Problem," *Machine Learning*, vol. 47, pp. 235–256, 2002.

[5] R. I. Brafman and M. Tennenholtz, "R-max: A General Polynomial Time Algorithm for Near-Optimal Reinforcement Learning," *Journal of Machine Learning Research*, vol. 3, pp. 213–231, 2002.